

MedLift-3D

Codebase Reference

Volumetric lung-nodule assessment from sparse-view and limited-angle chest projections

What this document is

A complete technical reference to the MedLift-3D repository: what the system takes in and produces, how the physics is modelled, what every file is for, how the important algorithms work, and why the design is shaped the way it is. It is written to be read either straight through or as a lookup — §4 and §5 are per-file entries.

At a glance

Purpose	Reconstruct 3-D chest anatomy and measure pulmonary nodule volumes from a handful of 2-D X-ray projections, for lower-dose lung-cancer screening and Volume Doubling Time assessment
Method	2-D diffusion prior alternated with hard data consistency (DiffusionMBIR structure), plus per-voxel posterior uncertainty
Size	~5,800 lines: 3,174 library · 1,738 scripts · 885 tests
Tests	9 correctness gate suites, 77 tests, ~35 s on CPU, no dataset required
Hardware target	One 12–16 GB GPU; Kaggle 12-hour sessions, offline; or a rented hourly box
Data	LIDC-IDRI (the only public source with voxel-level four-reader nodule contours), plus synthetic phantoms that need no download
Stack	PyTorch · NumPy · SciPy · scikit-image · nibabel · pandas

Contents

1 What MedLift-3D does

- 1.1 The input/output contract
- 1.2 The three stages
- 1.3 What is deliberately out of scope

2 The measurement problem

- 2.1 The forward model
- 2.2 Why it is hard: two acquisition tracks
- 2.3 The solution shape, and its two halves

3 Repository layout

- 3.1 The dependency direction

4 The core package, file by file

- 4.1 geometry.py
- 4.2 units.py
- 4.3 projector.py
- 4.4 phantom.py
- 4.5 datasets.py
- 4.6 prior2d.py
- 4.7 diffusion.py
- 4.8 solver.py
- 4.9 roi.py
- 4.10 gaussians.py
- 4.11 metrics.py
- 4.12 export.py, utils.py, logging_utils.py
- 4.13 baselines/

5 The scripts

- 5.1 run_gates.py
- 5.2 make_phantoms.py
- 5.3 prepare_lidc.py
- 5.4 train_prior.py
- 5.5 reconstruct.py
- 5.6 evaluate.py
- 5.7 mdvc.py
- 5.8 hallucination.py
- 5.9 ablate_roi.py
- 5.10 gpu_benchmark.py + remote.sh

6 The correctness gates

7 Configuration

8 End-to-end walkthrough

9 Design decisions under a single-GPU budget

10 Known limitations

11 Command reference

1 What MedLift-3D does

MedLift-3D reconstructs a three-dimensional volume of the chest, and measures the volume of pulmonary nodules inside it, from a **small handful of two-dimensional X-ray projections** rather than from a full CT scan. The clinical motivation is lung-cancer screening: a full CT delivers a radiation dose that limits how often a patient can be re-scanned, yet assessing whether a nodule is growing (its Volume Doubling Time) requires repeated measurement. Fewer projections means less dose.

1.1 The input/output contract

This is the single most important thing to be clear about, because every module in the repository is organised around it.

	What it is	Concrete example
Input	N projection images (2-D arrays of Beer–Lambert line integrals), plus the acquisition geometry that says where each ray went. No CT volume.	16 images of 271×383 px + a geometry record
Output	A 3-D volume in Hounsfield Units; a per-voxel uncertainty map; a nodule volume in mm ³ with a measurement report.	case.nii.gz case_uncertainty.nii.gz measurements.json
Ground truth	Used only to <i>train</i> the prior and to <i>score</i> the result. Never available at inference time.	mu (256 ³), nodule_mask

1.2 The three stages

Confusion usually comes from collapsing these together. They are separate programs, run at separate times, on separate data.

```

STAGE 1  TRAIN THE PRIOR           (once, offline, needs many CT volumes)
CT volumes -> 2-D axial slices -> diffusion model learns
                                   "what chest anatomy looks like"
scripts/train_prior.py             output: runs/prior/best.pt

STAGE 2  RECONSTRUCT              (per patient, needs only projections)
projections + geometry -> [ prior + data consistency ] -> volume
scripts/reconstruct.py             output: case.nii.gz, uncertainty map

STAGE 3  MEASURE                  (per nodule)
volume + nodule location -> segmentation -> volume in mm^3
scripts/evaluate.py, scripts/mdvc.py output: metrics, VDT input

```

Where do the projections come from?

There is no tomosynthesis scanner in this project. Projections are **simulated** from real CT volumes (LIDC-IDRI) or from synthetic phantoms, using the *same* forward operator the solver later optimises through. That keeps the physics on both sides of the loss identical. Real scanner data would introduce a domain gap that must be measured deliberately, not assumed away.

1.3 What is deliberately out of scope

- **Nodule detection.** The nodule location is an input to the measurement stage, supplied by the reading workflow. The project measures nodules; it does not find them.
- **Malignancy classification.** The output is a volume, not a diagnosis.
- **Absolute CT-quality volumetry from 15 views over 30°.** The limited-angle track (B) leaves a 150° missing wedge; what is claimed there is *relative* change detection with an uncertainty map, and the minimum-detectable-volume-change experiment quantifies exactly where that claim stops holding.

2 The measurement problem

2.1 The forward model

An X-ray travelling through tissue is attenuated exponentially. Taking the logarithm of the detected intensity turns that into a straight line integral of the **linear attenuation coefficient** μ (units: mm^{-1}) along the ray:

```
I = I0 * exp( -integral( mu dl ) )      Beer-Lambert
=> p = -log(I / I0) = integral( mu dl )  the "line integral" we work with

Discretised over all rays:  p = A mu    A is linear
```

Because A is **linear**, three things follow, and the codebase leans on all three:

- Its transpose A^T is exactly its vector–Jacobian product, so back-projection never needs a second hand-written kernel that could drift out of sync with the forward one (`projector.py`).
- The gradient of the data-fidelity term $\frac{1}{2}\|A\mu - p\|^2$ is exactly $A^T(A\mu - p)$ (`solver.py`).
- A volume can be split into background plus a region of interest and the projections split with it, *exactly* — which is what makes the two-scale refinement in `roi.py` valid rather than approximate.

2.2 Why it is hard: two acquisition tracks

A full CT collects roughly a thousand projections over 180° , which makes A well conditioned. This project collects one to two orders of magnitude fewer. The repository carries two settings side by side and reports them separately.

Track	Geometry	Views / arc	Detector at 256^3	Difficulty
A	Parallel-beam sparse-view CT	8–64 over 180°	271×383	Mildly ill-posed. Angular coverage is complete, just sparse. Comparable to the published literature. Do this first.
B	Limited-angle digital tomosynthesis	15–25 over 30°	411×575	Severely ill-posed: a 150° missing wedge means whole directions in Fourier space are never measured, so depth resolution is intrinsically poor.

Track A exists as a de-risking step: if the pipeline cannot reconstruct 32 views over 180° , it certainly will not manage 15 over 30° , and diagnosing the failure on the easy case is far cheaper.

2.3 The solution shape, and its two halves

With too few measurements, infinitely many volumes fit the data. Something has to choose among them. MedLift-3D follows **DiffusionMBIR** (Chung et al., CVPR 2023): a learned prior proposes anatomy, and the measurement operator forces the proposal to agree with what was actually measured. Neither half works alone:

Half	Supplies	Fails alone because
Diffusion prior (2-D, slice-wise)	Plausible chest anatomy, sharp edges, realistic texture	It has never seen <i>this</i> patient. On its own it generates a convincing chest that is not the one being scanned — the hallucination risk the audit in <code>scripts/hallucination.py</code> exists to quantify.
Data consistency (A , A^T)	Patient specificity: agreement with the measured projections	The problem is under-determined, so agreeing with the data leaves the unmeasured directions unconstrained — streaks and blur.
z-direction TV	Coherence between adjacent axial slices	A 2-D prior treats slices independently; without this coupling the volume flickers from slice to slice.

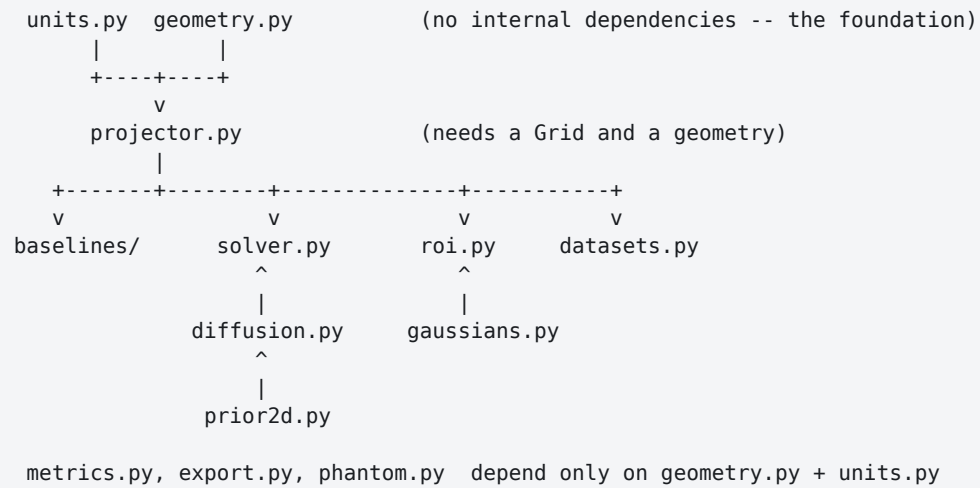
3 Repository layout

About 5,800 lines in three layers: a library that knows the physics, scripts that run experiments, and tests that refuse to let the library be silently wrong.

```
medlift3d/
|
+-- src/medlift3d/          THE LIBRARY (3,174 lines) -- physics, models, metrics
| |
| +-- geometry.py          174 Grid: a voxel array that knows where it is in space
| +-- units.py             74 HU <-> mu conversion; the one diagnostic window
| +-- projector.py         397 A and A^T: differentiable X-ray forward projection
| +-- phantom.py           254 synthetic chest phantoms (no dataset download needed)
| +-- datasets.py          359 cases on disk, patient-level splits, simulation
| +-- prior2d.py           191 the 2-D UNet noise estimator (48.1M params)
| +-- diffusion.py         228 DDPM training / DDIM sampling, config-locked
| +-- solver.py            219 prior + data consistency = reconstruction
| +-- roi.py               138 two-scale refinement around a nodule
| +-- gaussians.py         277 adaptive Gaussian parameterisation (the 05 ablation)
| +-- metrics.py           391 PSNR/SSIM, nodule Dice, volumetry, calibration
| +-- export.py            120 NIfTI, DICOM SEG/SR, triplanar PNG
| +-- utils.py             97 TV, seeding, checkpoints
| +-- logging_utils.py     55 offline-safe CSV logging (Kaggle has no wandb)
| +-- baselines/           182 fbp, sirt, sirt_tv, cgls -- what the prior must beat
|
+-- scripts/                EXPERIMENTS (1,738 lines) -- each writes CSV + figures
| +-- run_gates.py         run every correctness gate, print a pass/fail table
| +-- make_phantoms.py     generate a synthetic dataset with projections
| +-- prepare_lidc.py      ingest real chest CT onto the canonical grid
| +-- train_prior.py       train the 2-D diffusion prior (Kaggle-resumable)
| +-- reconstruct.py      run any method over a split, one output layout
| +-- evaluate.py          score reconstructions, write the comparison table
| +-- mdvc.py             minimum detectable volume change vs views and arc
| +-- hallucination.py     does the prior invent or erase nodules?
| +-- ablate_roi.py        objective 05: Gaussians vs plain voxels
| +-- gpu_benchmark.py     measure this GPU, then price the whole programme
| +-- remote.sh            drive a rented vast.ai box over SSH
|
+-- tests/                 NINE GATE SUITES (885 lines, 77 tests)
+-- configs/              geometry . prior . solver . roi (YAML, declarative)
+-- notebooks/            kaggle_pipeline.ipynb
+-- docs/                 REVIEW . CODE_AUDIT . PLAN . IMPLEMENTATION . REMOTE_GPU
```

3.1 The dependency direction

Nothing in the library imports from scripts/. Within the library the dependencies run one way, from physics outward, so a change to the metrics can never alter the forward model:



4 The core package, file by file

Each entry below gives the module's **purpose**, its **public API**, **how it works**, and — where the design is non-obvious — **why it is built that way**. The order follows the dependency graph in §3.1: foundations first.

4.1 geometry.py

174 lines · A voxel array that knows where it is in physical space

Purpose. The deepest failure mode in volumetric work is an array that has lost its physical frame: a reconstruction and its ground truth end up on different grids and every metric computed between them is meaningless — silently, with no exception raised. Grid makes the frame part of the type.

Symbol	What it does
<code>Grid(shape, spacing, origin)</code>	Frozen dataclass. Index order is always (z, y, x) ; origin is the world position of the <i>centre</i> of voxel [0,0,0], in mm, RAS.
<code>grid.affine</code>	4×4 index → world matrix in (x, y, z) NIfTI order.
<code>grid.world_to_norm(pts)</code>	World mm → the [-1, 1] coordinates <code>F.grid_sample</code> expects. This is the bridge between physics and PyTorch sampling.
<code>grid.assert_matches(other)</code>	Raises <code>AssertionError</code> on any shape or affine mismatch. Called before every comparison in <code>evaluate.py</code> .
<code>Grid.spacing_from_affine(a)</code>	Extracts spacing from column norms , not <code>np.diag</code> — the latter is wrong for any oblique acquisition.
CHEST, ROI	The two canonical grids: 256 ³ at 1.5 mm, and 128 ³ at 1.0 mm. Anything else in a reconstruction path is a bug.
<code>roi_grid_at(centre_xyz)</code>	A fine grid centred on a world point, e.g. a nodule centroid.

How it works. Everything is derived from three tuples. `extent_mm` is the span from first to last voxel *centre* (not the outer bounding box), which is precisely the convention `align_corners=True` uses — so world ↔ normalised ↔ index conversions all agree to floating-point precision. `tests/test_geometry.py` asserts that agreement at the corners.

4.2 units.py

74 lines · One quantity, one unit, enforced everywhere

Purpose. The quantity every model predicts, every projector integrates and every loss compares is the linear attenuation coefficient μ in mm^{-1} — non-negative and **unbounded above**. Mixing μ , Hounsfield Units and a normalised [0,1] field is a classic silent scale error, so all conversion lives here.

Symbol	What it does
<code>MU_WATER = 0.0193</code>	Water at ~70 keV effective energy, mm^{-1} .
<code>HU_CLIP = (-1000, 400)</code>	The single diagnostic window. Imported by preprocessing, simulation and evaluation alike so they cannot drift apart.
<code>MU_MAX = 0.02702</code>	μ at the top of the window. The physical clamp.
<code>hu_to_mu / mu_to_hu</code>	The affine map, with clipping, so $\mu \geq 0$.

Symbol	What it does
NODULE_MU_THRESHOLD	-300 HU in μ : solid nodules against -820 HU parenchyma.
apply_poisson(p, i0)	Photon noise applied correctly — in <i>intensity</i> space, then transformed back: $I = I_0 e^{-p}$, $\tilde{I} \sim \text{Poisson}(I)$, $p \approx -\log(\tilde{I}/I_0)$.
mu_to_net / net_to_mu	$\mu \rightarrow [-1, 1]$ for the network.

Why the [-1, 1] clamp is legitimate here

Clamping a network output to $[-1, 1]$ is normally arbitrary. Here it is not: μ is bounded below by 0 (air) and above by MU_MAX (the top of the diagnostic window), so the clamp is *exactly* a physical constraint. The rule that follows: never apply a bounded activation such as sigmoid to μ itself, and never re-normalise a reconstruction with the HU affine map — that collapses a density-unit volume to a near-constant field.

4.3 projector.py

397 lines · A and A^T -- the most important module in the repository

Purpose. Differentiable X-ray forward projection, built entirely from torch operations so that autograd supplies the backward pass. There is no opaque library call in the middle of the graph to sever it.

Symbol	What it does
ParallelGeometry	Sparse-view CT: <code>n_views</code> equispaced over <code>arc_deg</code> , rotating about <code>z</code> . Endpoint excluded, since over 180° the last view would duplicate the first.
DTSGeometry	Limited-angle tomosynthesis: the source translates along <code>x</code> , the detector is stationary. <code>arc_deg</code> is the angle subtended <i>at the object</i> , the conventional quotation.
<code>.covering(grid, n_views, ...)</code>	Sizes the detector from the grid , with margin. A detector that under-covers the object truncates the line integrals and no reconstruction method can recover from that, so it is computed, never hardcoded.
<code>Projector(grid, geometry)</code>	Precomputes every ray once. Sampling is chunked (<code>chunk_rays</code>) so peak memory stays bounded regardless of detector size.
<code>proj.fp(mu) -> [V, nv, nu]</code>	Forward project. Differentiable in μ .
<code>proj.bp(projs) -> [nz, ny, nx]</code>	Adjoint back-projection A^T .
<code>proj.grad_data_term(mu, p)</code>	$A^T(A\mu - p)$, the true gradient of $\frac{1}{2}\ A\mu - p\ ^2$.
<code>proj.coverage()</code>	Fraction of rays that hit the volume. Near 1.0 means an oversized detector; very low means it is mis-aimed.
<code>proj.benchmark_bp()</code>	Times both adjoint implementations on <i>this</i> hardware and reports which wins.

How the forward projection works — trilinear ray sampling (Joseph's method):

```

1 Build rays      each detector pixel + view angle -> (origin, direction)
2 Clip to the box slab-method ray/AABB intersection -> (t_near, t_far)
                  rays that miss get dt = 0 and contribute nothing
3 Sample         S points along each ray, S ~ diagonal / min(spacing)
                  i.e. roughly one sample per voxel -- the standard rate
4 Interpolate     F.grid_sample(volume, points, align_corners=True)
                  trilinear, zero-padded outside the volume
5 Integrate       sum the samples * dt -> the line integral

```

How the adjoint works, and why it cannot be wrong

Because A is linear, its vector-Jacobian product is A^T . The default `bp_vjp` therefore calls `.backward()` on the forward pass rather than running a second hand-written kernel — which makes an fp/bp inconsistency **structurally impossible**. Gradients accumulate into one `probe.grad` buffer across chunks, so peak memory is a single volume regardless of ray count. A hand-written `bp_scatter` is kept as a tested alternative; measured on CPU it runs several times slower than `grid_sample`'s fused backward kernel, so the choice is benchmarked rather than assumed.

bp is NOT filtered back-projection

`Projector.bp` is the plain adjoint. FBP is A^T composed with a ramp filter and is a *reconstruction* operator. Substituting one for the other gives wrong gradient updates that still look like they are converging, so the two live in separate modules: filtered back-projection is in `baselines/fbp.py`.

4.4 phantom.py

254 lines · Synthetic chest phantoms, so the pipeline is runnable with no download

Purpose. Real work uses LIDC-IDRI, but every gate test, the solver and a smoke training run must be executable with no dataset download — which is what makes the pipeline verifiable *before* any data agreement is signed and before any GPU time is bought.

Symbol	What it does
<code>make_chest_phantom(grid, seed)</code>	Returns $(\mu, \text{nodule_mask}, \text{meta})$. Builds torso, muscle shell, lungs, spine, ribs, tapering vessels and nodules in HU, then converts once to μ .
<code>make_water_cylinder(grid, r)</code>	A uniform water cylinder. Used by gate G3: the peak line integral through it must equal $\mu_{\text{water}} \times 2r$.
<code>grow_nodule(grid, mu, n, gain)</code>	Re-renders one nodule with its volume scaled by $(1 + \text{gain})$. This is how the MDVC experiment is built, since LIDC has no follow-up scans.
<code>erase_nodule(grid, mu, n)</code>	Inpaints a nodule out of the ground truth. If the prior puts it back, that is a measured hallucination.
<code>lung_mask(mu, grid, fill=True)</code>	Hole-filled lung region. Restricts PSNR/SSIM and bounds the nodule search.

Two details that are load-bearing. First, nodules get a proper **partial-volume edge** (occupancy in $[0,1]$, mask at $\geq 50\%$); a hard binary sphere would make volumetry artificially easy and hide exactly the error the project is trying to measure. Second, a shrinking free mask enforces **keep-out zones** between nodules: touching nodules segment as one blob, which makes per-nodule volumetry ill-posed for reasons that have nothing to do with reconstruction quality.

Why lung_mask fills holes

A raw air-density threshold (−950 to −400 HU) *excludes* nodules and vessels, because they are denser than parenchyma. Two things break if you stop there: `psnr_lung` gets computed everywhere *except* the structures the project is about, and a nodule does not lie inside its own lung mask — so restricting a segmentation to the mask deletes it. Filling makes this an anatomical region rather than a density band, which is what standard lung segmentation produces.

4.5 datasets.py

359 lines · Cases on disk, patient-level splits, simulation, and the slice cache

Symbol	What it does
<code>default_dirs()</code>	Auto-detects Kaggle (/kaggle/input read-only, /kaggle/working writable) versus local paths.
<code>save_case / load_case</code>	One .npz per case carrying μ , its grid, the nodule mask, every projection track with its geometry, and optional conditioning volumes.
<code>simulate(mu, grid, geom, i0)</code>	Forward-projects with the same operator the solver optimises through , then applies Poisson noise.
<code>make_split / verify_split</code>	Deterministic patient-level split; <code>verify_split</code> raises on any overlap.
<code>CaseDataset</code>	Whole cases, for reconstruction and evaluation.
<code>SliceDataset</code>	Axial slices for training the 2-D prior, backed by a memory-mapped slice cache.

Why splits are checked and not trusted. Data leakage here does not produce a visibly wrong number — it produces a flatteringly right one. If the prior has seen a test patient, hallucination becomes indistinguishable from reconstruction and the entire safety argument collapses. So `verify_split` is a hard assertion called at the top of every script that touches a split.

The slice cache

The obvious implementation — open the case .npz and index one slice — costs a full decompression of the volume *per slice*, because `mmap_mode` is silently ignored for zipped archives. Measured at 357 ms for a 256^3 case, that is roughly 72 minutes of pure I/O per epoch. `SliceDataset` therefore builds one contiguous .npy of the kept slices on construction and memory-maps it, turning random access into a page fault:

```
per random 256^2 slice: 0.074 ms (was 357 ms) ~4800x
epoch I/O, 60 cases: ~1 s (was ~72 min)
disk cost: ~3 GB for 60 cases x 200 slices, float32
```

The cache key hashes the case list, fields and slice shape, so it is rebuilt only when the dataset changes. If the data directory is read-only (Kaggle inputs), it falls back to the run directory; if nothing is writable it degrades to direct reads rather than failing.

4.6 prior2d.py

191 lines · The 2-D axial-slice UNet -- 48.1M parameters

Purpose. The noise estimator at the heart of the diffusion prior. A 2-D prior rather than a 3-D one is **the decision that makes this project fit on a single 12–16 GB GPU**: roughly 7 GB and a day of training instead of 16–40 GB and a week, and it removes a 3-D autoencoder from the critical path entirely.

Symbol	What it does
UNetConfig	in_ch, cond_ch, base_dim=64, dim_mults=(1,2,4,8), attn_at=(2,3).
UNet2D(cfg)	Encoder/decoder with skip connections; returns the same spatial shape it is given.
ResBlock	Two convolutions with scale-shift timestep conditioning; the second convolution is zero-initialised so each block starts as the identity.
SelfAttention2d	Applied only at the two coarsest levels — attention is $O(N^2)$ in pixels.
SinusoidalPosEmb	Timestep embedding.

3-D coherence without a 3-D network. Two mechanisms supply it. The physics does most of the work: rays cross slices, so the projector couples them whether the network knows it or not. An explicit z-direction total-variation term in the solver suppresses the residual slice-to-slice flicker.

Conditioning: unconditional by default, and why

cond_ch = 0 by default. If a conditioning channel is used it must carry something that *also exists at inference* — a CGLS or FBP initialisation, stored in the dataset and named with --cond-key. Conditioning on the clean target teaches the network to copy its conditioning channel: training loss looks excellent, and at inference the network faithfully copies the blurry initialisation instead. The dataset layer makes the clean target unreachable as a conditioning source, and the solver reads cond_ch from the checkpoint rather than trusting its own config.

4.7 diffusion.py

228 lines · DDPM training and DDIM sampling, with the config locked to the weights

Symbol	What it does
DiffusionConfig	Frozen dataclass: timesteps, schedule (cosine linear), objective (eps x0), loss_type, cfg_drop_prob, x0_clip. Validated in __post_init__.
q_sample(x0, t, noise)	The forward noising process.
loss(x0, cond)	Trains the network, with classifier-free guidance dropout.
eps_to_x0 / x0_to_eps	The two conversions, exact inverses of each other.
predict_eps(x, t, cond, g)	Model output converted to ϵ , with optional classifier-free guidance.
ddim_sample(..., x0_hook)	DDIM sampling. The solver injects data consistency through x0_hook.
state() / load_state(ck)	Checkpoint I/O. load_state refuses a config mismatch.
from_checkpoint(ck)	Rebuilds both the UNet and the wrapper from the checkpoint's own configs, so mismatched arguments cannot be passed by hand.

The silent bug this module is shaped to prevent

Train a network to predict *noise*, then have the sampler interpret its output as the *clean signal*. Nothing crashes. Loss curves look normal. A thousand sampling steps later the reconstruction is garbage. The defence is structural, not documentary: the config is frozen, written into every checkpoint, and load_state raises rather than loading a checkpoint whose objective, schedule or timestep count differs from the sampler being built. There is no code path that lets training and sampling disagree.

4.8 solver.py

219 lines · Where the prior meets the measurements -- the reconstruction itself

The reverse-diffusion loop, one step:

```
eps <- prior(x_t, t, cond)           # 2-D, batched over z slices
x0  <- (x_t - sqrt(1 - a_t) * eps) / sqrt(a_t)   # denoise to an estimate

for j in 1..M:                       # DATA CONSISTENCY
  g  <- c * A^T(A * net_to_mu(x0) - p)         # c = MU_MAX / 2
  g  <- g + lam_z * scale * dTV_z(x0)          # inter-slice coupling
  x0 <- x0 - lr * g                          # lr = dc_step/(c^2 L)

x0  <- clamp(x0, physical range)
x_{t-1} <- ddim_step(x_t, x0, eps)
```

Symbol	What it does
SolverConfig	n_steps=50 DDIM steps, dc_steps=2, dc_step=0.9, lam_z=0.03, guidance, eta, slice_batch=16, n_posterior=8, warm_start_t=0.7.
estimate_lipschitz(proj)	Largest eigenvalue L of $A^T A$ by power iteration.
DiffusionSolver.sample_once	One posterior sample.
DiffusionSolver.reconstruct	K samples → mean (the reconstruction) and per-voxel standard deviation (the uncertainty map).

Three design points worth understanding

- **The step size is derived, not tuned.** L comes from power iteration on $A^T A$, so one dc_step value behaves consistently across grid sizes, view counts and geometries instead of needing a retune per experiment. The chain rule must be applied to the step size as well as to the gradient: with $\mu = c \cdot x + \text{const}$, the Hessian in network units is $c^2 A^T A$, so the stable step is $\text{dc_step} / (c^2 L)$. Dropping the c^2 under-relaxes by a factor of about 5,500 — the data term then runs every iteration and does essentially nothing, which is not visible from the code or the loss.
- **Warm start.** The trajectory begins partway down the schedule ($\text{warm_start_t}=0.7$) seeded from a classical CGLS reconstruction, rather than from pure noise. Fewer steps, and the measurements steer from the first iteration.
- **The uncertainty map is nearly free.** K posterior samples at 256^3 in float32 is about 270 MB. Their standard deviation is the only thing that lets a reader tell measured structure from structure the prior invented — which, for a method built on a generative prior, is the difference between a usable result and an unfalsifiable one.

4.9 roi.py

138 lines · Two-scale refinement: an exact decomposition, not an approximation

Purpose. A 1 mm whole-chest grid is unaffordable on a 16 GB card, and unnecessary: nodules are at most ~30 mm across and the reading workflow already localises them. So reconstruct the chest coarsely, then refine a small fine-grid box around the nodule.

```
mu      = mu_bg      + mu_roi      split the volume
A mu    = A_chest(mu_bg) + A_roi(mu_roi) A is LINEAR, so this is exact
residual r = p - A_chest(mu_bg)          what the ROI must explain

then fit  A_roi(mu_roi) ~ r    on a fine grid, at a fraction of the cost
```

Symbol	What it does
<code>resample(mu, src, dst)</code>	Trilinear resample between world-aligned grids.
<code>zero_roi(mu, chest, roi)</code>	Copy of μ with the ROI's world extent zeroed.
<code>background_residual(...)</code>	$r = p - A(\mu_{bg})$.
<code>refine_roi(..., use_gaussians)</code>	The refinement optimisation. <code>use_gaussians=False</code> runs the identical optimisation on plain voxels — the control arm for objective O5.

The decomposition introduces **no approximation** beyond resampling the background at the box boundary. `tests/test_roi_exact.py` asserts the same-grid case is exact and *measures* the fine-grid resampling error rather than assuming it away.

4.10 gaussians.py

277 lines · Anisotropic Gaussians as an adaptive basis -- and a falsifiable claim

Purpose. An adaptive, edge-aware parameterisation of a voxel field. Gaussians are evaluated onto the ROI grid and the resulting volume is line-integrated by the ordinary projector, so the physics is identical to every other stage.

Be precise about what this is not

This is **not** a splat rasteriser and it does **not** make rendering real-time or memory-free — the dense ROI volume is materialised on every iteration. Gaussians cannot be justified here on efficiency grounds. The claim they *can* support is narrower: that they are a compact *adaptive* basis. Whether that actually helps is an empirical question, and `scripts/ablate_roi.py` is built to answer it either way. A negative result is a finding: it would mean the ROI stage should be plain voxels and 3DGS belongs in the viewer, if anywhere.

Symbol	What it does
<code>GaussianField(grid, cfg)</code>	Parameters per primitive: position, raw amplitude, log scale (3 axes), rotation quaternion.
<code>field.amp</code>	softplus , so attenuation is non-negative and unbounded . A sigmoid would cap μ at 1.0 mm^{-1} and saturate its own gradient.
<code>init_from_volume(mu)</code>	Seeds primitives with probability $\mu + \text{grad } \mu $, covering both bulk tissue and edges.
<code>calibrate_amplitude(mu)</code>	One closed-form least-squares factor $\langle r, \mu \rangle / \langle r, r \rangle$ removes the overlap bias (measured $\sim 6\times$) that seeding each amplitude from the local μ introduces.
<code>rasterize()</code>	Evaluates the sum onto a voxel grid in a local window per primitive. Differentiable.
<code>regularisation()</code>	L1 on σ (suppresses needle artefacts) and an anisotropy penalty (fights missing-wedge elongation in Track B). Acts on the primitives , never on a rasterised dense volume.
<code>prune(min_amp, optimizer)</code>	Drops dead primitives and re-binds the optimizer , slicing its Adam moments to match.

Why prune takes the optimizer. Pruning replaces every `nn.Parameter`. An optimizer already holding the old tensors then goes on stepping detached buffers while the live parameters never move again — silently. In a 800-iteration refinement that prunes every 200 steps, three quarters of the optimisation would do nothing, and the O5 ablation would be rigged against Gaussians without anything looking wrong.

4.11 metrics.py

391 lines · Metrics that can actually detect a difference

Two rules drive this module.

- **Nodule metrics are computed inside a dilated bounding box, never over the whole volume.** Thresholding a whole chest at a soft-tissue level selects the body wall, the spine and every vessel, so a whole-volume Dice against a few-hundred-voxel nodule mask is pinned near zero regardless of reconstruction quality — it carries no information.
- **PSNR and SSIM are restricted to the lung mask.** Whole-image PSNR on a chest CT is dominated by air, which every method reconstructs perfectly, so it flatters them all equally and discriminates nothing.

Symbol	What it does
<code>global_metrics(pred, gt, lung)</code>	PSNR, 3-D SSIM (not per-slice averaged) and MAE in HU, inside and outside the lung mask.
<code>nodule_metrics(...)</code>	Dice and volume error for one nodule. Returns two families — see below.
<code>adaptive_threshold(roi)</code>	Midpoint of local background and nodule peak, which recovers the 50%-occupancy iso-surface.
<code>measure_nodule_volume(...)</code>	Volume in mm ³ at a location, using no ground truth . The measurement used by the MDVC experiment.
<code>hallucinated_at(...)</code>	Did a nodule-like blob appear where the ground truth has none?
<code>calibration(samples, gt)</code>	Empirical coverage of the posterior interval, plus its expected calibration error.
<code>relative_volume_change(a, b)</code>	Percent change — the quantity Volume Doubling Time is derived from.

Why two families of nodule number

Metric family	Question it answers	Property
<code>nodule_dice,</code> <code>volume_ape_pct</code>	How much error did <i>reconstruction</i> introduce?	The same segmentation operator is applied to the reconstruction and to the ground-truth μ . Segmentation bias cancels, so a perfect reconstruction scores 1.0 / 0%. These compare methods.
<code>dice_vs_consensus,</code> <code>ape_vs_consensus_pct</code>	How close is the result to the reference contour?	Compared against the reader-consensus mask. Includes the segmenter's own partial-volume bias. These describe absolute clinical agreement.

Reporting only the second family conflates two error sources; reporting only the first hides the absolute floor. The evaluation reports both.

Three constraints keep volumetry well-posed. Detection and measurement use *different* thresholds — adaptive thresholding presupposes a nodule is present and will segment noise into something, so detection gates on the fixed clinical threshold first. A **geometric bound** confines candidates to a ball of `max_diameter_mm/2`, because at a fixed soft-tissue threshold a nodule near the pleura connects to the chest wall and the component containing the seed can otherwise become the whole body. An optional **anatomical bound** (the hole-filled lung mask) severs that connection properly. A fourth requirement — that the selected component actually touch the seed — stops a nearby vessel fragment being returned as a measurement after a nodule has been erased.

4.12 export.py, utils.py, logging_utils.py

272 lines · Output, shared numerics, and offline-safe logging

Symbol	What it does
<code>export.save_nifti</code>	Writes a volume with its affine intact. Without it a viewer renders the wrong aspect ratio and every downstream measurement is wrong.
<code>export.save_uncertainty</code>	The per-voxel posterior standard deviation, in μ rather than HU.
<code>export.save_dicom_seg</code>	DICOM Segmentation object via <code>highdicom</code> ; returns <code>None</code> when the dependency or the source instances are absent, so the pipeline degrades rather than fails.
<code>export.write_measurement_report</code>	Volume measurements as JSON, mirroring what a DICOM SR would carry. Written unconditionally.
<code>export.save_triplanar_png</code>	Axial / coronal / sagittal preview with optional mask contour.
<code>utils.tv</code> , <code>tv_grad</code> , <code>tv_grad_z</code>	Isotropic total variation and its analytic gradient, written out rather than obtained by autograd because the solver calls them inside <code>no_grad</code> blocks many times per step.
<code>utils.seed_everything</code>	Python, NumPy, torch and CUDA seeds together.
<code>logging_utils.CsvLogger</code>	Appends metrics to a CSV. Kaggle notebooks have no internet, so wandb is not an option — and a killed 12-hour session leaves its history behind for the next one to continue.

4.13 baselines/

182 lines · What a learned prior has to beat to justify its existence

Method	File	How it works	Note
FBP	<code>fbp.py</code>	Ramp-filter each detector row, then back-project.	Carries a discretisation-dependent normalisation, so <code>estimate_fbp_scale</code> recovers it once per (grid, geometry) by reconstructing a disc of known attenuation.
SIRT	<code>iterative.py</code>	Simultaneous iterative reconstruction with row and column preconditioning and a box constraint.	No calibration constant.
SIRT-TV	<code>iterative.py</code>	SIRT with interleaved total-variation descent.	The strong classical baseline for sparse-view CT. <code>tv_weight</code> is relative to the data update, so it transfers across grid sizes.
CGLS	<code>iterative.py</code>	Conjugate gradients on the normal equations.	Fast and calibration-free, so it is the default initialiser for the diffusion solver. Semi-convergent — keep the iteration count modest.

5 The scripts

Every script shares the same conventions: `argparse` with defaults from `default_dirs()` so the same command works on Kaggle and locally; a printed header stating device, method and case count; raw results to CSV plus a summary table to stdout; and `verify_split` called before any split is used.

5.1 run_gates.py

61 lines

Run this first, always. Executes the nine correctness gate suites as separate `pytest` processes and prints a pass/fail table. Each gate guards a failure mode that is *silent* — it produces plausible loss curves and wrong reconstructions — so they are checked before any GPU time is spent, not after.

```
python scripts/run_gates.py          # ~35 s on CPU, no data needed
```

Writes: nothing; exit code 0 only if all nine suites pass.

5.2 make_phantoms.py

104 lines

Generates a synthetic chest dataset with simulated projections for every track. This is the route that makes the whole pipeline runnable with no dataset download, which matters on an offline Kaggle image. Real data goes through `prepare_lidc.py`; everything downstream is identical.

```
python scripts/make_phantoms.py --n-cases 40 --out data/phantom
python scripts/make_phantoms.py --n-cases 40 --shape 256 256 256 \
    --views-a 16 32 --views-b 15 --arc-b 30
```

Writes: one `.npz` per case, `splits.csv`, `dataset.json`.

5.3 prepare_lidc.py

236 lines

Ingests real chest CT onto the canonical grid. Two routes, because a full LIDC-IDRI download (~124 GB) is not feasible on Kaggle: **(A)** LIDC-IDRI through `pylidc`, which gives voxel-level nodule contours from four independent readers ($\geq 50\%$ consensus); **(B)** a directory of volumes already on disk, which gives no masks — usable for training the prior, not for the clinical metrics. LIDC-IDRI is the only public source with voxel-level contours; the LUNA family ships centroids only, from which no segmentation ground truth can be built.

```
python scripts/prepare_lidc.py --source pylidc --limit 400 --out data/lidc
python scripts/prepare_lidc.py --source dir --in /kaggle/input/luna16 \
    --out data/lidc
```

Writes: the same case layout as the phantom route, so every downstream script is unchanged.

Coordinate frames on the real-data route

SimpleITK reports LPS; this project and NIfTI use RAS. Converting means negating the x and y world axes — which, with spacing kept positive, requires *reversing the array* along those axes and taking the far corner as the new origin. Negating the origin alone leaves a volume that is mirrored left–right and anterior–posterior relative to the frame it claims. That is self-consistent inside the pipeline and wrong the moment laterality matters — an exported NIfTI, or the question of which lung a nodule is in.

5.4 train_prior.py

190 lines

Trains the 2-D axial-slice diffusion prior. Built for a 12-hour Kaggle session: `--max-hours` stops cleanly before the notebook is killed, and rerunning the same command resumes from `last.pt` automatically (optimizer and AMP scaler state included). Validation uses **fixed** noise and timesteps per batch index, because resampling them every epoch makes the curve so noisy that "best" gets chosen by luck.

```
python scripts/train_prior.py --data data/phantom --out runs/prior \
    --epochs 60 --batch-size 8 --max-hours 11

# conditional variant -- only with a conditioning volume in the dataset
python scripts/train_prior.py --cond-key cgls ...
```

Writes: `best.pt`, `last.pt`, `log.csv` and a loss-curve PNG.

5.5 reconstruct.py

153 lines

Reconstructs a split with any method, writing every method to the same layout so `evaluate.py` can compare them without special cases. Reconstructions always live on the case's own grid. For the diffusion method the prior is rebuilt from the checkpoint's own configs, so mismatched arguments cannot be passed by hand.

```
python scripts/reconstruct.py --track A16 --method sirt_tv
python scripts/reconstruct.py --track A16 --method diffusion \
    --prior runs/prior/best.pt --n-posterior 8
```

Writes: per case: `.npz` ($\mu + \text{std}$), `.nii.gz`, `_uncertainty.nii.gz`, `.png`; plus `settings.json` recording every argument.

5.6 evaluate.py

117 lines

Scores reconstructions against ground truth and writes the comparison table. Calls `assert_matches` on every pair before comparing. Results are stratified by nodule size, because small nodules are the hard, clinically decisive case and averaging over all sizes hides exactly where a method fails. Reports PSNR/SSIM/MAE and both nodule metric families — not FID or MMD, which measure distributional similarity for unconditional generation whereas this is a paired per-patient task with ground truth for every case.

```
python scripts/evaluate.py --recon runs/recon --data data/phantom
```

Writes: `global.csv`, `nodules.csv`, and summary tables to stdout.

5.7 mdvc.py

185 lines

Minimum detectable volume change as a function of view count and arc — the experiment that answers the clinical objective directly. Volume Doubling Time needs a *change* in volume to be measurable and LIDC has no follow-up scans, so growth is synthesised on the ground truth, the projections are re-simulated, and the identical measurement is applied to baseline and follow-up. The output curve says where absolute volumetry stops being trustworthy.

```
python scripts/mdvc.py --tracks A32 A16 B15 --gains 0.05 0.1 0.25 0.5
```

Writes: mdvc_raw.csv, mdvc_summary.csv, mdvc.png, and the MDVC threshold per (method, track).

5.8 hallucination.py

152 lines

The safety experiment. Under sparse or limited-angle acquisition the reconstruction is under-determined, so a learned prior *must* invent structure — and nodules are exactly the small high-frequency features a prior is most likely to invent or smooth away. Three controlled conditions per nodule: **present** (should be detected → sensitivity), **erased** from the ground truth (must *not* appear → false-positive rate), and **inserted** (should survive the pipeline → insertion recall).

```
python scripts/hallucination.py --tracks A16 B15 \
    --methods sirt_tv diffusion --prior runs/prior/best.pt
```

Writes: hallucination_raw.csv, a summary pivot, and sensitivity stratified by nodule size.

5.9 ablate_roi.py

141 lines

Objective O5. Runs the identical two-scale ROI refinement twice — once with an adaptive Gaussian field, once on a plain voxel grid — from the same coarse reconstruction at matched iteration count, and reports nodule volume error, RMSE against ground truth and wall-clock for each. Deliberately falsifiable.

```
python scripts/ablate_roi.py --track A16 --iters 600
```

Writes: ablate_roi_raw.csv, ablate_roi_summary.csv, and a printed verdict.

5.10 gpu_benchmark.py + remote.sh

399 lines

Run `gpu_benchmark.py` **first on any rented box**, before launching anything long. It measures forward/adjoint time, training-step time and peak VRAM on the real operations, re-benchmarks the two adjoint implementations, and prices the whole experimental programme at a given hourly rate — turning "how long and how much" from a guess into a measurement while the meter is running. `remote.sh` drives a vast.ai instance over SSH: `setup`, `sync`, `launch` (detached under `tmux`, output tee'd to a log), `watch`, `fetch`, `cost`.

```
python scripts/gpu_benchmark.py --rate 0.40

export MEDLIFT_HOST='root@1.2.3.4 -p 40000'
./scripts/remote.sh setup && ./scripts/remote.sh sync
./scripts/remote.sh launch train 'python scripts/train_prior.py ...'
./scripts/remote.sh watch train
```

Writes: a timing and cost table; log files under `logs/` on the remote box.

6 The correctness gates

Nine test suites, 77 tests, about 35 seconds on a CPU with no dataset. Every one of them guards a failure mode that produces **plausible loss curves and wrong results** — which is why they are gates rather than tests, and why nothing should be trained until all nine pass.

Gate	File	Asserts	Guards against
G1	test_geometry.py 10 tests	One canonical grid; affine round-trips; spacing from column norms; <code>assert_matches</code> rejects mismatches.	Comparing volumes that live on different grids — every metric between them is then meaningless.
G2	test_adjoint.py 8 tests	$\langle Ax, y \rangle = \langle x, A^T y \rangle$; <code>fp</code> is differentiable and has a <code>grad_fn</code> ; both adjoint implementations agree; descending $A^T(Ax - p)$ really reduces the residual.	A projection loss with no gradient , so the regulariser becomes the entire objective and converges very stably to smooth mush.
G3	test_units.py 7 tests	$HU \leftrightarrow \mu$ round-trips; air is zero attenuation; the peak line integral through a water cylinder equals $\mu_{\text{water}} \cdot 2R$.	HU/density confusion, spacing errors, ray-step normalisation errors and detector-scale errors — all caught by one assertion.
G4	test_metrics.py 12 tests	A <i>perfect</i> reconstruction scores perfectly; degradation lowers scores; metrics are local, not whole-volume; erased nodules measure zero; growth is monotonic.	Metrics that cannot detect anything — pinned near a constant regardless of quality, or charging reconstruction for the segmenter's own bias.
—	test_no_leakage.py 4 tests	Splits are disjoint, deterministic, and survive a CSV round trip.	Leakage, which does not produce a visibly wrong number but a flatteringly right one.
—	test_diffusion_config.py 12 tests	Objective/schedule/timestep mismatches are refused on load; $\epsilon \leftrightarrow x_0$ conversions are inverses; the prior is unconditional by default; the clean target is unreachable as a conditioning source.	Training on one objective and sampling on another; or training on conditioning that does not exist at inference.
—	test_roi_exact.py 5 tests	<code>zero_roi</code> partitions the volume; the same-grid decomposition is exact; resampling error is measured.	A two-scale scheme that quietly introduces error it then attributes to the reconstruction.
—	test_gaussians.py 8 tests	Quaternions give real rotations; amplitude is non-negative and unbounded; rasterisation is differentiable; calibration reduces error; pruning keeps the optimizer alive .	A sigmoid capping μ and saturating its gradient; or an optimizer stepping detached buffers after a prune, freezing one arm of the ablation.
—	test_solver.py 5 tests	Data consistency measurably drives the projection residual down ; it is stable for $dc_step < 2$; <code>L</code> bounds the operator; the solver declines to condition an unconditional prior.	A data term that runs every iteration and does nothing — present in the code, inert in effect, invisible in the loss.

7 Configuration

Four declarative YAML files. `geometry.yaml` is the **single source of truth** for acquisition: the simulator and the projector both read it, so they cannot drift apart into different view counts.

File	Holds	Values worth knowing
geometry.yaml	Grid, ROI, both tracks, detector, physics	grid 256^3 @ 1.5 mm; ROI 128^3 @ 1.0 mm; Track A parallel, 180° , 8/16/32/64 views; Track B DTS, 30° , 15/25 views, src 1000 mm, det 400 mm; $i_0 = 10^5$ photons; HU clip $[-1000, 400]$.
prior.yaml	UNet and diffusion hyperparameters, training budget	base_dim 64, dim_mults (1,2,4,8), attention at the two coarsest levels, cond_ch 0 ; 1000 timesteps, cosine schedule, objective eps ; batch 8, lr $1e-4$, AMP on, max_hours 11.
solver.yaml	Reconstruction and baselines	50 DDIM steps (not 1000 — that buys nothing and costs a 20–40× slowdown); dc_steps 2; dc_step 0.9; lam_z 0.03; warm_start_t 0.7; n_posterior 8.
roi.yaml	Two-scale refinement and the O5 ablation	800 iterations; 150k Gaussians; window 7; per-parameter learning rates; lam_scale $1e-4$, lam_aniso $1e-3$; arms matched on iterations.

8 End-to-end walkthrough

A complete run on synthetic data, with the shapes that actually appear. Substituting `prepare_lidc.py` for `make_phantoms.py` changes nothing downstream.

```

[0] GATES ~35 s, CPU
python scripts/run_gates.py -> 9/9 suites passed

[1] DATA minutes, CPU/GPU
python scripts/make_phantoms.py --n-cases 40 --shape 256 256 256

each case .npz holds
mu (256, 256, 256) float32 0.0 .. 0.027 mm^-1
nodule_mask (256, 256, 256) uint8 labels 0,1,2,3
proj_A16 (16, 271, 383) float32 line integrals, Poisson noise
proj_B15 (15, 411, 575) float32
geom_A16, geom_B15, meta, shape, spacing, origin
splits.csv: train 30 / val 5 / test 5 (patient-level, committed)

[2] PRIOR ~9 h on one 24 GB GPU, AMP
python scripts/train_prior.py --data data/phantom --out runs/prior \
--epochs 60 --batch-size 8 --max-hours 11
-> runs/prior/best.pt (48.1M)

[3] RECONSTRUCT per case, per method
python scripts/reconstruct.py --track A16 --method sirt_tv
python scripts/reconstruct.py --track A16 --method diffusion \
--prior runs/prior/best.pt --n-posterior 8

runs/recon/diffusion_A16/
phantom0035.nii.gz reconstruction, HU, opens in 3D Slicer
phantom0035_uncertainty.nii.gz per-voxel posterior std
phantom0035.npz mu + std + grid
phantom0035.png triplanar preview
settings.json every argument, for provenance

[4] SCORE
python scripts/evaluate.py --recon runs/recon --data data/phantom
global.csv psnr, psnr_lung, ssim, ssim_lung, mae_hu_lung, uncertainty
nodules.csv nodule_dice, volume_ape_pct, *_vs_consensus, detected
stratified by <6mm / 6-10mm / 10-20mm / >20mm

[5] THE THREE EXPERIMENTS
python scripts/mdvc.py --tracks A32 A16 B15 # clinical claim
python scripts/hallucination.py --tracks A16 B15 # safety claim
python scripts/ablate_roi.py --track A16 # objective 05

```

Inside one reconstruction. `reconstruct.py` loads the case, builds a Projector on the case's own grid, and hands it to `DiffusionSolver`. The solver runs CGLS for an initialisation, estimates the Lipschitz constant by power iteration, then draws K independent posterior samples — each one a warm-started DDIM trajectory alternating the slice-wise prior with data-consistency gradient steps. Their mean is the reconstruction; their standard deviation is the uncertainty map.

9 Design decisions under a single-GPU budget

Every significant choice in this codebase was made against one constraint: **it must train and run on a single 12–16 GB GPU**, in sessions short enough for Kaggle's 12-hour cap or cheap enough to rent by the hour. The column that matters is the last one.

Decision	Alternative rejected	Why
2-D slice prior + z-TV coupling	3-D diffusion prior, or a 3-D latent autoencoder	~7 GB and a day, versus 16–40 GB and a week. Also removes a 3-D autoencoder from the critical path — 64× compression destroys structures the size of a nodule.
50 DDIM steps	1000 DDPM steps	Buys nothing measurable and costs a 20–40× inference slowdown.
Warm start at $t = 0.7$	Start from pure noise	Fewer steps, and the measurements steer the trajectory from the first iteration.
256³ @ 1.5 mm	512 ³	LIDC chest CT is 512×512×(100–400) with anisotropic spacing. A 512 ³ claim would require inventing the z resolution it purports to validate.
Two-scale ROI at 1 mm	1 mm over the whole chest	Unaffordable, and unnecessary: nodules are ≤ 30 mm and the reading workflow localises them. The decomposition is exact because A is linear.
Adjoint by VJP	A second hand-written back-projection kernel	An fp/bp inconsistency becomes structurally impossible. Chunked accumulation keeps peak memory at one volume.
Memory-mapped slice cache	Index slices out of the compressed .npz	357 ms per slice versus 0.074 ms — about 72 minutes of pure I/O per epoch, against a ~9 hour training budget.
Unconditional prior	Condition on an initialisation	Data consistency supplies patient specificity. A conditioning channel is only safe when the same thing exists at inference; otherwise the network learns to copy it.
Resume from checkpoint	Assume the session finishes	Kaggle kills sessions at 12 hours. - -max-hours stops cleanly and a rerun continues; a killed run costs nothing.
CSV logging	wandb or TensorBoard	Kaggle notebooks are offline by default. The CSV also survives a killed session for the next one to append to.

10 Known limitations

Stated here because a method built on a generative prior is only as credible as its declared failure modes.

- **Projections are simulated, not acquired.** There is no tomosynthesis scanner in this work. Simulation uses the same monochromatic operator the solver optimises through; a real scanner adds beam hardening, scatter and detector response. That domain gap is real and is not quantified here.
- **Track B cannot support absolute volumetry.** A 150° missing wedge means whole directions in Fourier space are never measured. What is claimed there is relative change detection with uncertainty, and `mdvc.py` measures where even that stops holding.
- **Juxtavascular nodules remain ambiguous.** A nodule fused to a vessel of the same density is genuinely hard for threshold-based volumetry. Dedicated vessel-aware segmentation is out of scope.

- **The prior can invent structure.** This is intrinsic to the approach, not a defect to be engineered away. It is why every reconstruction ships an uncertainty map and why the hallucination audit is a first-class experiment rather than an appendix.
- **Nodule location is an input.** Detection is assumed, supplied by the reading workflow.
- **Route B of the LIDC importer has no masks.** Volumes without contours can train the prior but cannot support Dice or volume error.

11 Command reference

SETUP

```
pip install torch numpy scipy scikit-image nibabel pandas pyyaml tqdm pytest matplotlib
export PYTHONPATH=src # or rely on the sys.path shim in each script
```

THE HAPPY PATH

```
python scripts/run_gates.py # 1. always first
python scripts/make_phantoms.py --n-cases 40 --out data/phantom
python scripts/train_prior.py --data data/phantom --out runs/prior --max-hours 11
python scripts/reconstruct.py --track A16 --method sirt_tv
python scripts/reconstruct.py --track A16 --method diffusion \
    --prior runs/prior/best.pt --n-posterior 8
python scripts/evaluate.py --recon runs/recon --data data/phantom
```

THE EXPERIMENTS

```
python scripts/mdvc.py --tracks A32 A16 B15 --gains 0.05 0.1 0.25 0.5
python scripts/hallucination.py --tracks A16 B15 --methods sirt_tv diffusion \
    --prior runs/prior/best.pt
python scripts/ablate_roi.py --track A16 --iters 600
```

TESTS

```
pytest tests/ -q # all 77
pytest tests/test_adjoint.py -q # one gate
```

RENTED GPU

```
python scripts/gpu_benchmark.py --rate 0.40 # measure before you spend
./scripts/remote.sh setup | sync | launch | watch | fetch | cost
```

USEFUL FLAGS

```
--device cuda|cpu --limit N --no-png --seed N
--shape 192 192 192 --n-posterior 4 --n-steps 30 # cheaper reconstruction
```

Further reading, in the repository: docs/REVIEW.md (defects found in the original design), docs/CODE_AUDIT.md (the four showstoppers in the previous codebase and how each is now unreachable), docs/PLAN.md (objectives O1–O5 and the schedule), docs/IMPLEMENTATION.md (build order and gates), docs/REMOTE_GPU.md (the rented-GPU runbook).